

■ Day 03 — REST APIs, CRUD & Backend Architecture

Backend fundamentals: REST APIs, MVC structure, HTTP methods, status codes & Mongoose CRUD

1. What is a REST API?

REST (Representational State Transfer) is an architectural style for building APIs. A REST API lets the frontend and backend communicate over HTTP using a fixed set of methods (GET, POST, PUT, PATCH, DELETE). Each request is stateless — the server doesn't remember previous requests, so every request must carry all the information needed to process it.

How frontend and backend talk: the frontend (React, mobile app, or Postman) sends an HTTP request to a specific URL (endpoint). The backend server receives it, processes the logic, talks to the database if needed, and sends back a response — usually in JSON format — along with a status code indicating success or failure.

2. Routes vs Controllers vs Models

This is the MVC-style separation used in most Node.js/Express backends:

- **Route** — defines the API endpoint (the URL + HTTP method) and points it to the correct controller function. Example: `router.get('/users/:id', getUser)`
- **Controller** — contains the actual business logic: reading request data, talking to the model, deciding what response to send back.
- **Model** — represents the database schema and handles all direct interaction with the database (via Mongoose in a MongoDB app).

3. HTTP Methods

- **GET** — retrieve/read data. No body needed. Safe and idempotent.
- **POST** — create a new resource. Data sent in the request body.
- **PUT** — update/replace an entire resource with new data.
- **PATCH** — partially update a resource (only the fields provided).
- **DELETE** — remove a resource.

req.body, req.params, req.query

- **req.body** — data sent by the client in the request body (used in POST, PUT, PATCH). Example: form data or JSON payload like `{ name: 'John' }`.
- **req.params** — dynamic values captured from the URL path. Example: in `/users/:id`, `req.params.id` gives the id from the URL.
- **req.query** — key-value pairs from the URL's query string. Example: `/users?age=25` → `req.query.age` is 25.

4. HTTP Status Codes

- **200 OK** — request succeeded (used for successful GET, PUT, PATCH, DELETE).
- **201 Created** — a new resource was successfully created (used after POST).
- **400 Bad Request** — the request is invalid or missing required data.
- **401 Unauthorized** — the user is not authenticated (no/invalid login token).
- **403 Forbidden** — the user is authenticated but not allowed to access this resource.
- **404 Not Found** — the requested resource/route doesn't exist.
- **409 Conflict** — the request conflicts with the current state (e.g. duplicate email on signup).
- **500 Internal Server Error** — something went wrong on the server side (unhandled error, bug, DB failure).

5. Mongoose CRUD Methods

- **create()** — inserts a new document into the collection. Used for POST requests.
- **find()** — retrieves multiple documents matching a filter (or all documents if no filter is given).
- **findById()** — retrieves a single document by its MongoDB `_id`.
- **findByIdAndUpdate()** — finds a document by id and updates it in one step. Used for PUT/PATCH.
- **findByIdAndDelete()** — finds a document by id and removes it. Used for DELETE.

try/catch and Error Handling

Every controller function that talks to the database or does async work should be wrapped in try/catch. If something fails (bad data, DB down, invalid id), the catch block sends a proper error response with the right status code instead of crashing the server.

6. API URL Structure

A clean REST URL structure usually follows the pattern: `/api/<resource>/<id>`. Examples: GET `/api/users` (list all), GET `/api/users/:id` (get one), POST `/api/users` (create), PUT `/api/users/:id` (update), DELETE `/api/users/:id` (delete). Resource names are plural nouns, not verbs — the HTTP method already tells you the action.

7. Why We Separate Routes, Controllers and Models

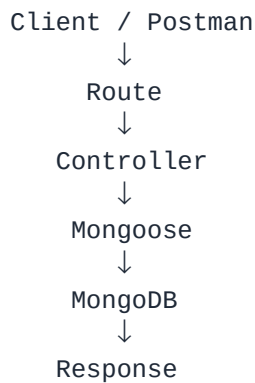
This separation follows the **separation of concerns** principle. It keeps the codebase organized, testable, and easy to scale — routes only deal with endpoints, controllers only deal with logic, and models only deal with data. This also makes it easier for teams to work in parallel and debug issues faster since each layer has one clear responsibility.

8. Testing APIs Using Postman

Postman is a tool used to send test requests (GET, POST, PUT, PATCH, DELETE) to your API without needing a frontend. You can set headers, add a JSON body, check the response status code

and data, and save requests in collections — making it essential for backend development and debugging before the frontend is even connected.

■ Day 03 Project Flow



■ Interview-ready Answer

"I separate my backend into routes, controllers and models. Routes define the API endpoints, controllers contain the business logic, and Mongoose models handle database operations. I use appropriate HTTP methods and status codes depending on the operation."

■ Day 03 Key Takeaways

Concept	One-liner
REST API	Allows systems to communicate through HTTP
Route	Defines an API endpoint
Controller	Contains request-handling/business logic
Model	Handles database interaction
req.body	Data sent in request body
req.params	Dynamic values from URL
CRUD	Create, Read, Update, Delete
Postman	Tool for testing APIs